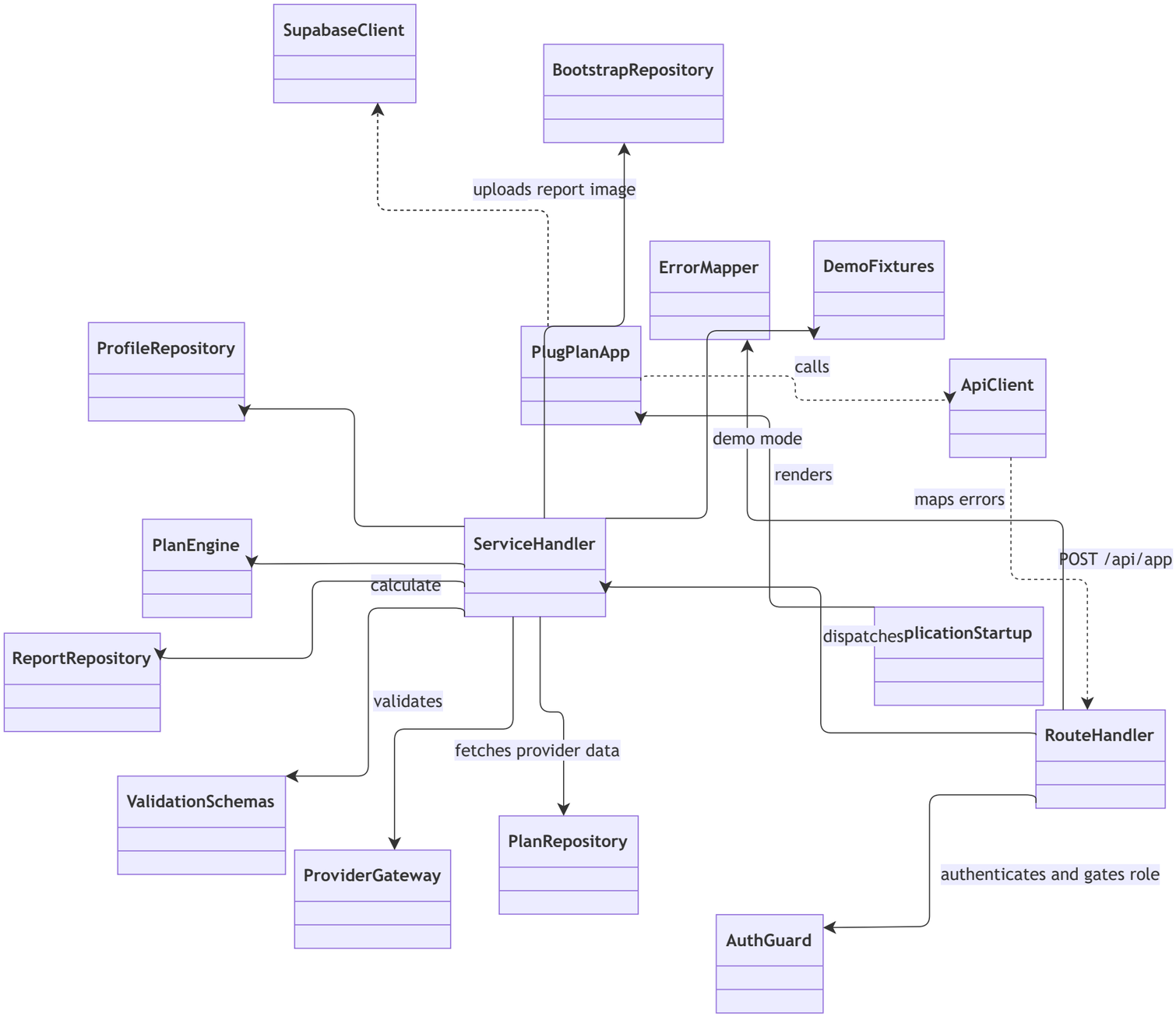
Design Class Model

Overview - boundary, control and data access

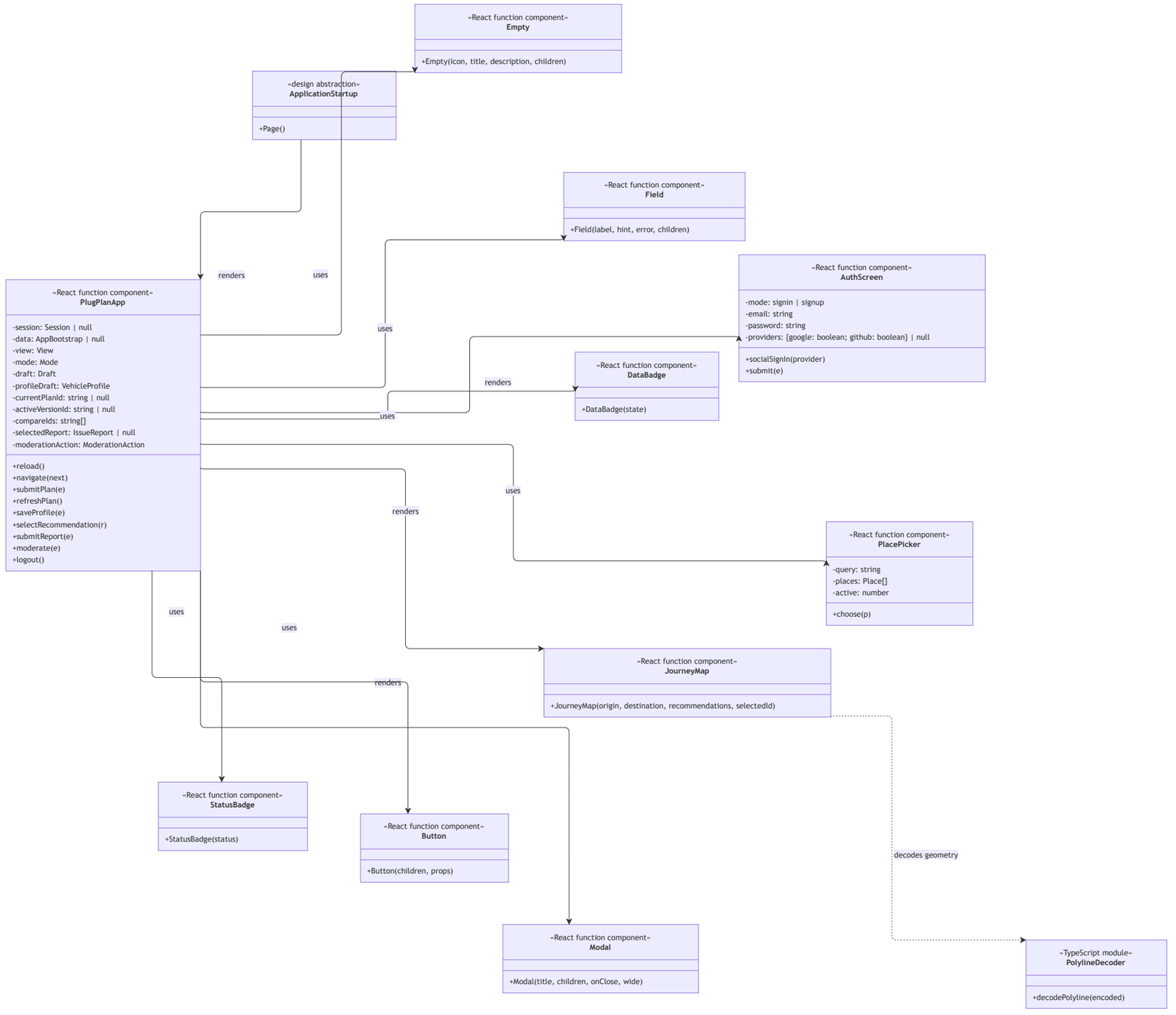
Dependencies across the detailed class views. Attributes and operations follow on the next six sheets.



Design Class Model

1. Presentation and startup

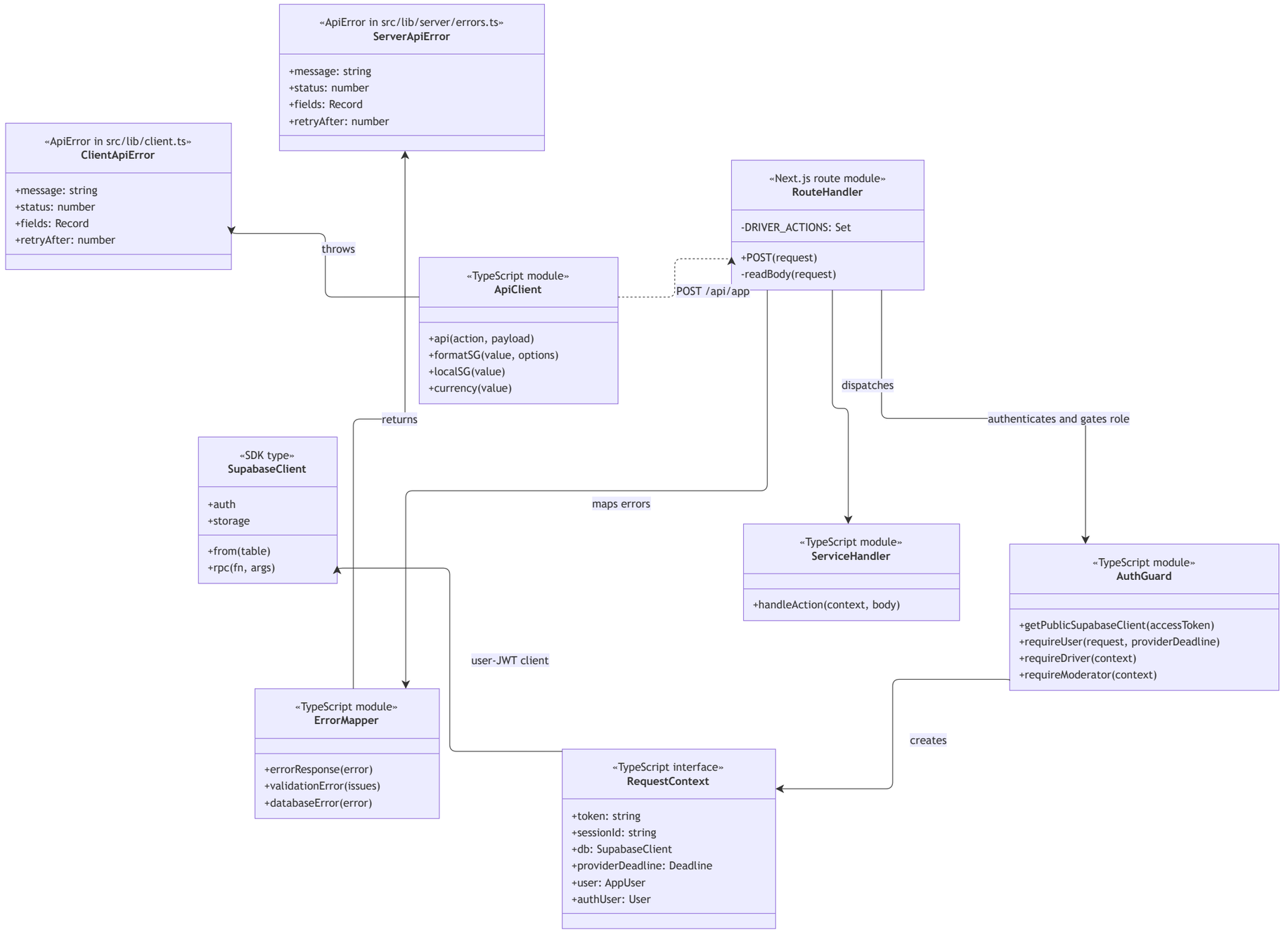
PlugPlan SG | SCE3-04 | Compartments show attributes and operations; cross-view dependencies appear in the source model.



Design Class Model

2. API and application control

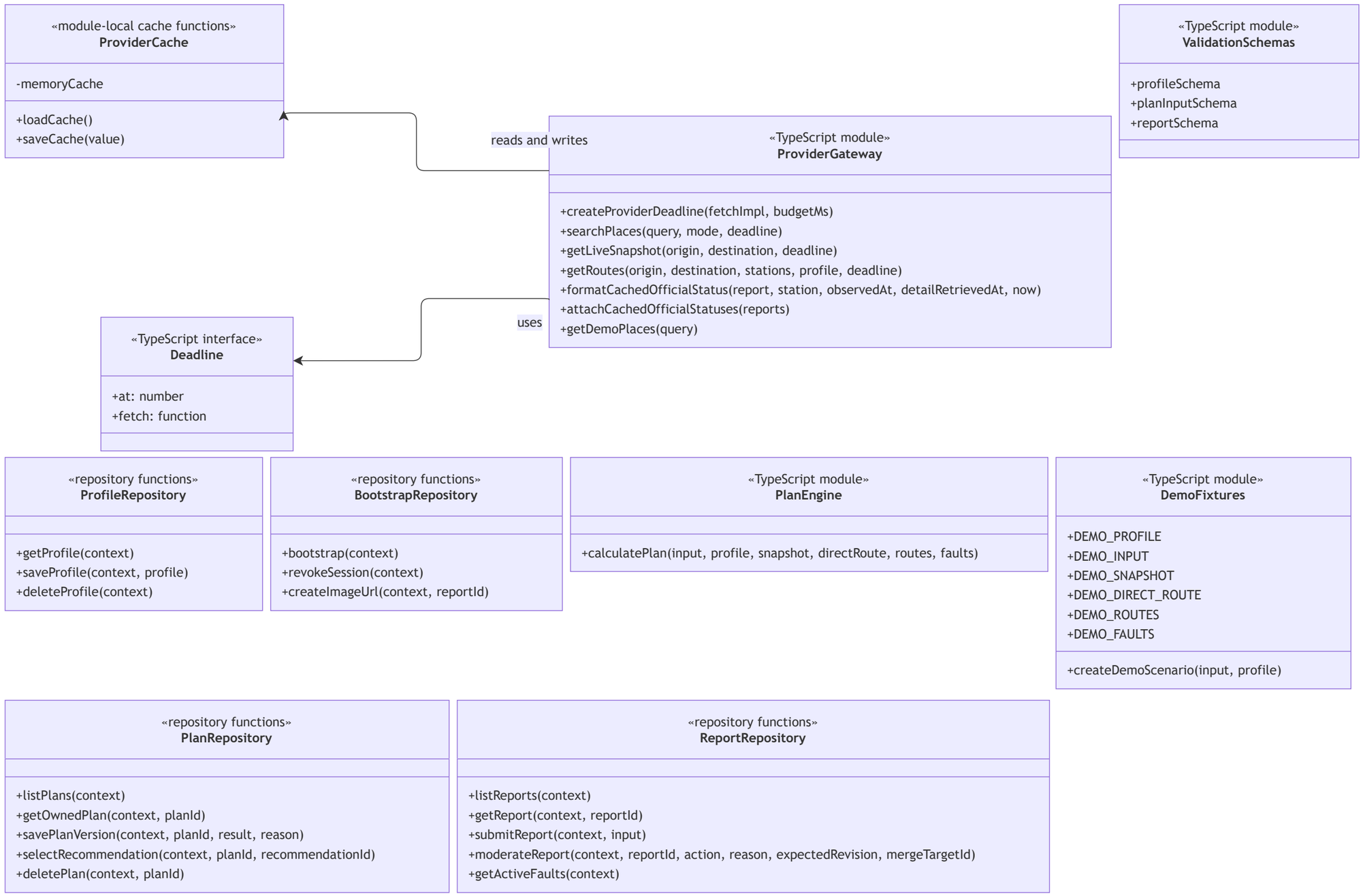
PlugPlan SG | SCE3-04 | Compartments show attributes and operations; cross-view dependencies appear in the source model.



Design Class Model

3. Calculation, providers and repositories

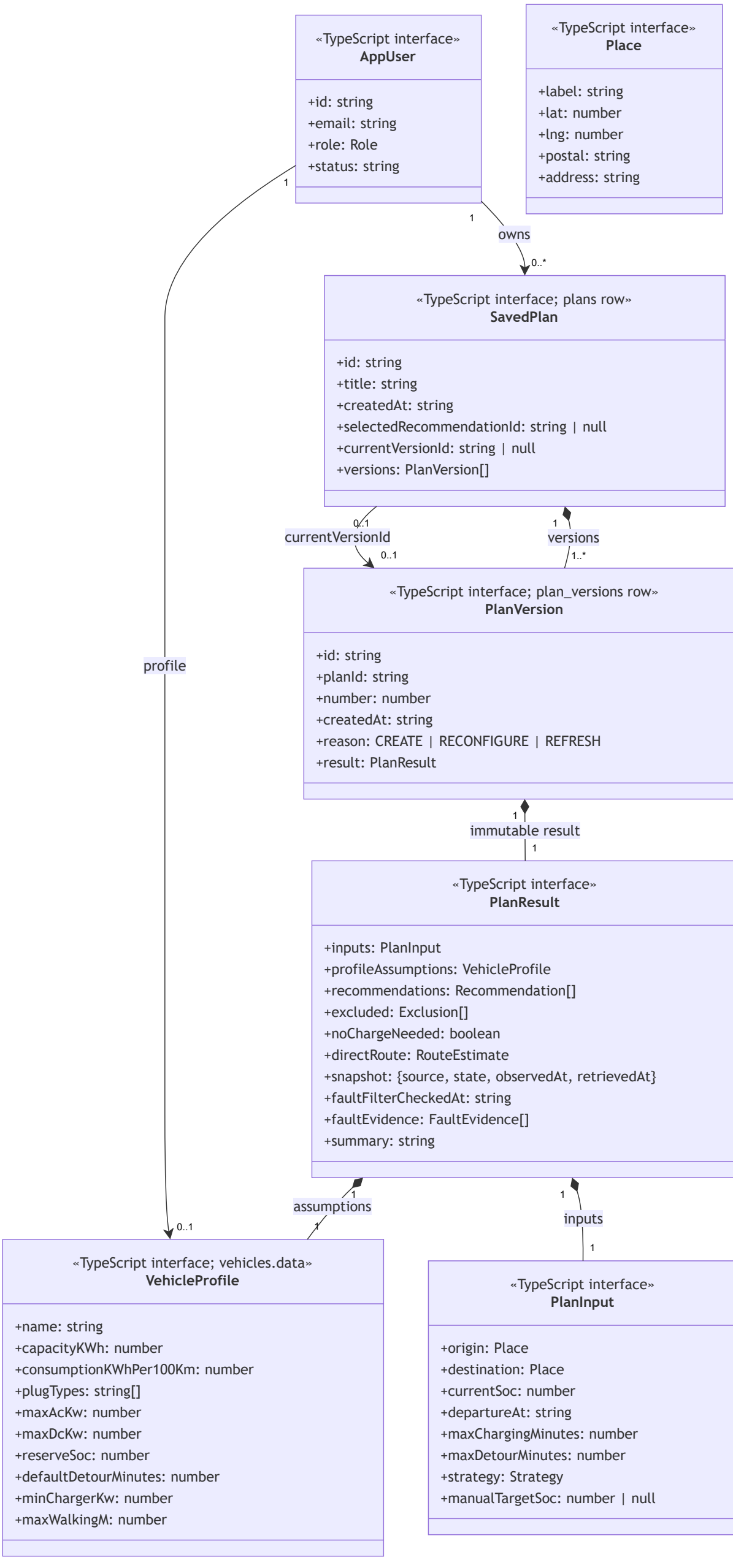
PlugPlan SG | SCE3-04 | Compartments show attributes and operations; cross-view dependencies appear in the source model.



Design Class Model

4. Plan ownership and immutable versions

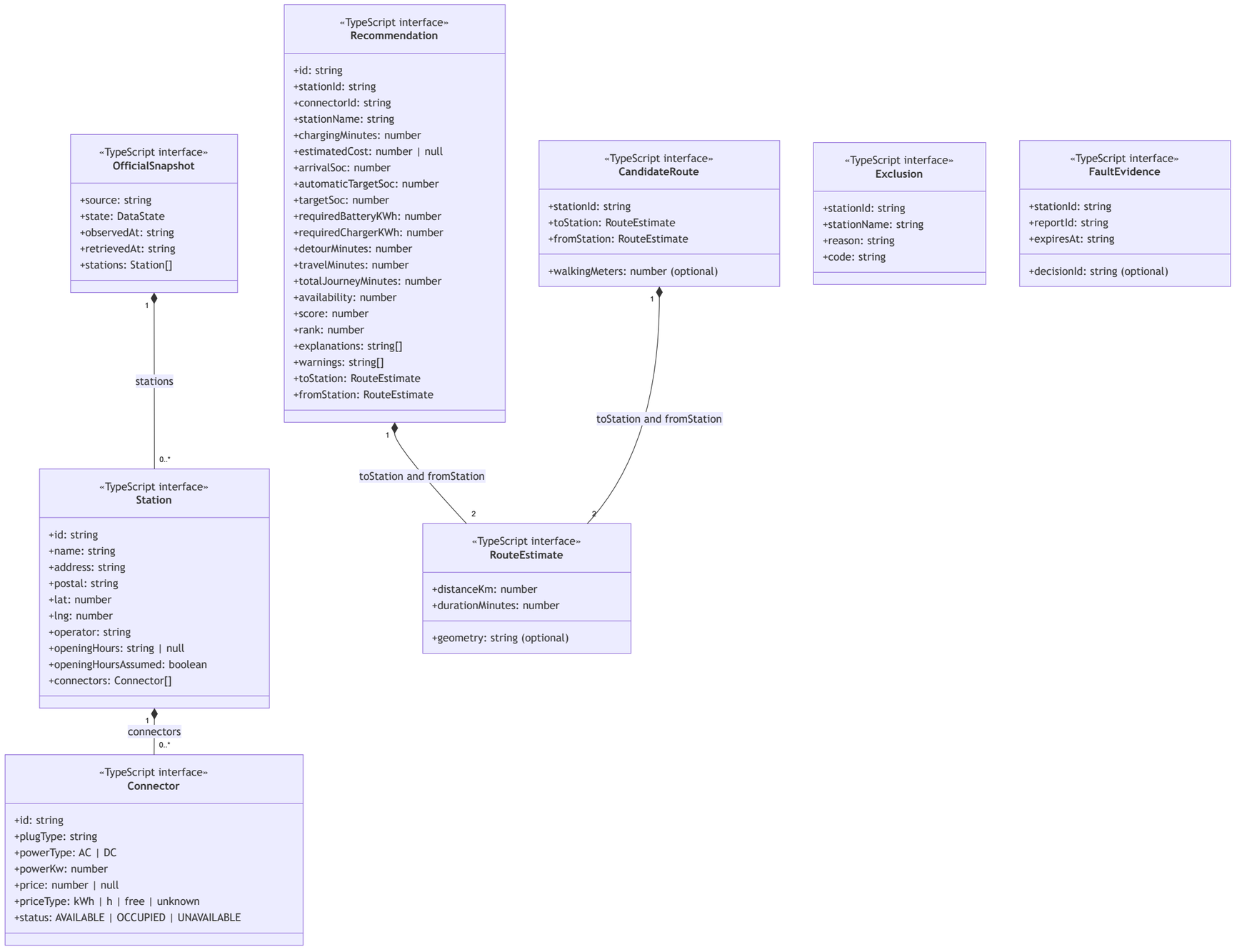
PlugPlan SG | SCE3-04 | Compartments show attributes and operations; cross-view dependencies appear in the source model.



Design Class Model

5. Station and calculation evidence

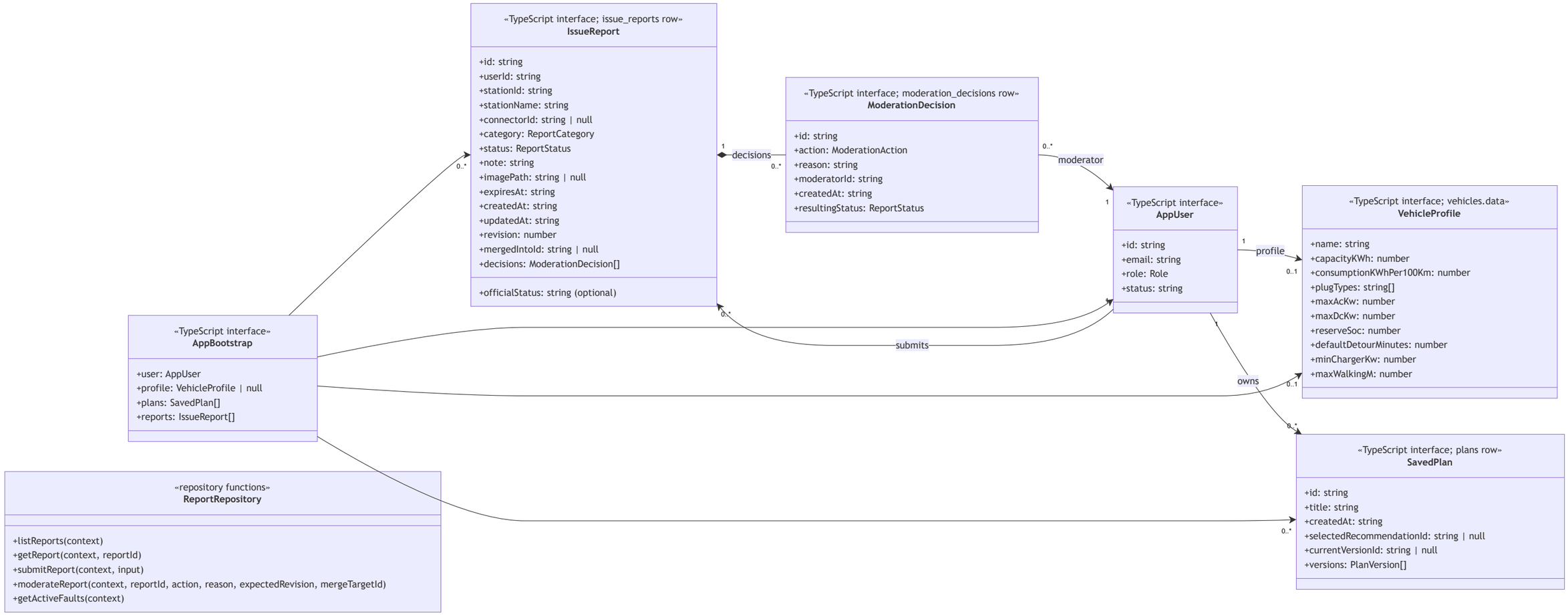
PlugPlan SG | SCE3-04 | Compartments show attributes and operations; cross-view dependencies appear in the source model.



Design Class Model

6. Reports, audit and result containers

PlugPlan SG | SCE3-04 | Compartments show attributes and operations; cross-view dependencies appear in the source model.



PlugPlan SG — Class Diagram

UML mapping and design notes

This is a design view of the Lab 3 implementation. React function components, TypeScript modules, SDK objects, and TypeScript interfaces are marked with stereotypes; only ApiError is a project class in the browser client, and the server has a separate class with the same name. Operations on modules/components name their actual exported functions or component functions. ApplicationStartup is a design abstraction for app startup, mapped to src/app/page.tsx's Page() and PlugPlanApp's session/bootstrap effects.

The main patterns are layered client–server coordination (handleAction), repository-style functions in src/lib/server/db.ts, provider gateway/cache functions in providers.ts, and strategy-based ranking in calculatePlan. Browser sign-in and report-image upload use the public Supabase client directly. On API requests, requireUser validates the Supabase user, active-session RPC, and app_users row; the route applies the driver/moderator role gate, and database calls keep the user's JWT so RLS remains in force.

Persistence mapping: a driver's optional VehicleProfile is stored in vehicles.data; a SavedPlan owns immutable PlanVersion history, with currentVersionId pointing to one version. selectedRecommendationId is an optional pointer into that current version and is checked by the database trigger. Reports persist category and status (there is no type field); officialStatus is computed from the provider cache and is not stored. OAuth UI is conditional on Supabase provider configuration and is not enabled in the baseline.